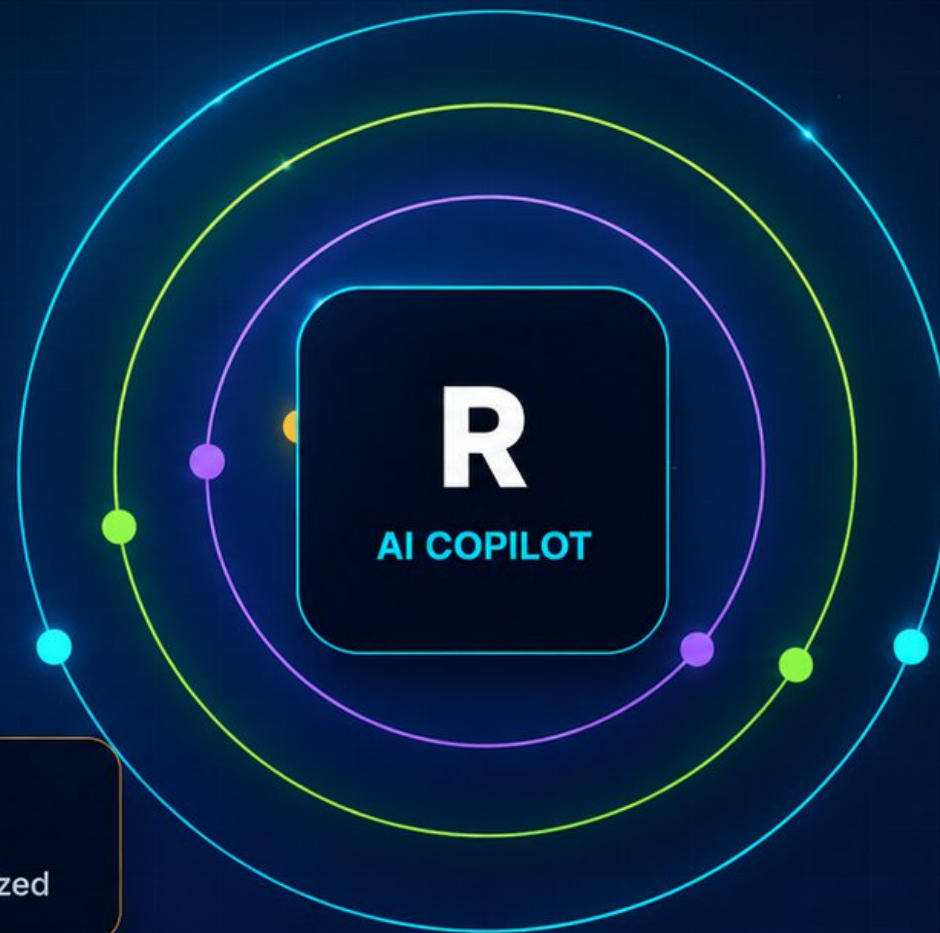


DETAILED ARCHITECTURE STORY

Recovera AI Copilot

Hybrid RAG + SQL backend for governed
revenue leakage analytics.



8

agents

32

tables

280

columns

4

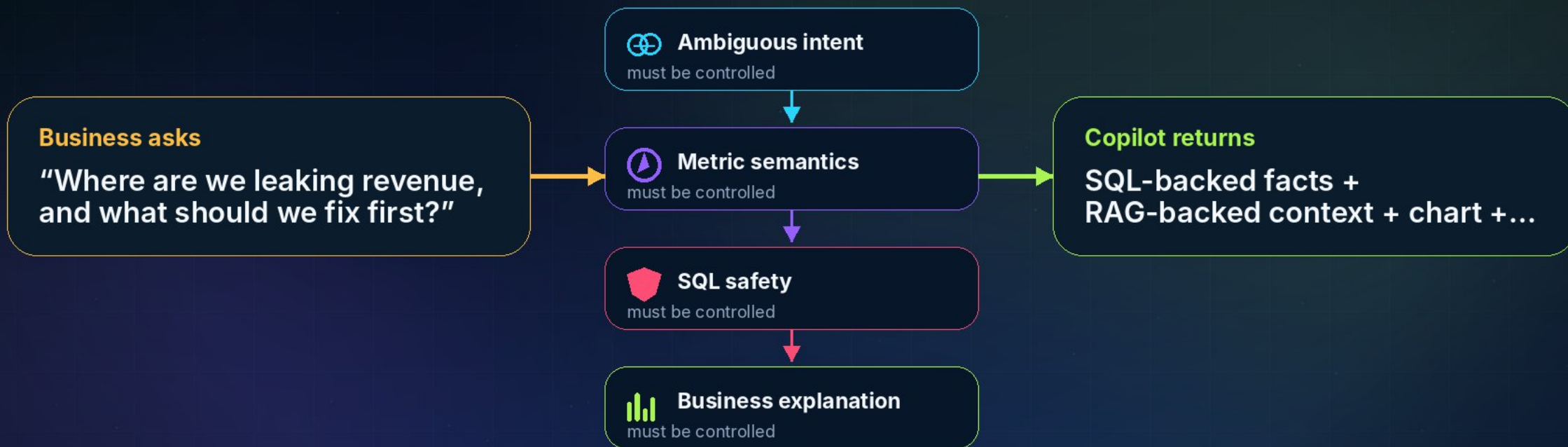
materialized
views



Mohamed Waleed Ezat

DATA & AI | ARCHITECTURE | ANALYTICS

Why the backend exists



BUSINESS

What it gives the business

Detect leakage



Find revenue at risk by city, seller, product, scenario and time.

Explain root cause



Use RAG over reviews, rules and schema docs to explain the why.

Prioritize actions

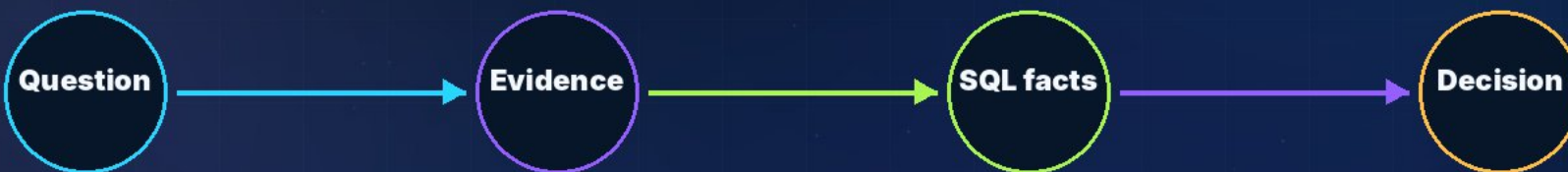


Rank risks by financial impact, rate, volume or campaign value.

Make it usable



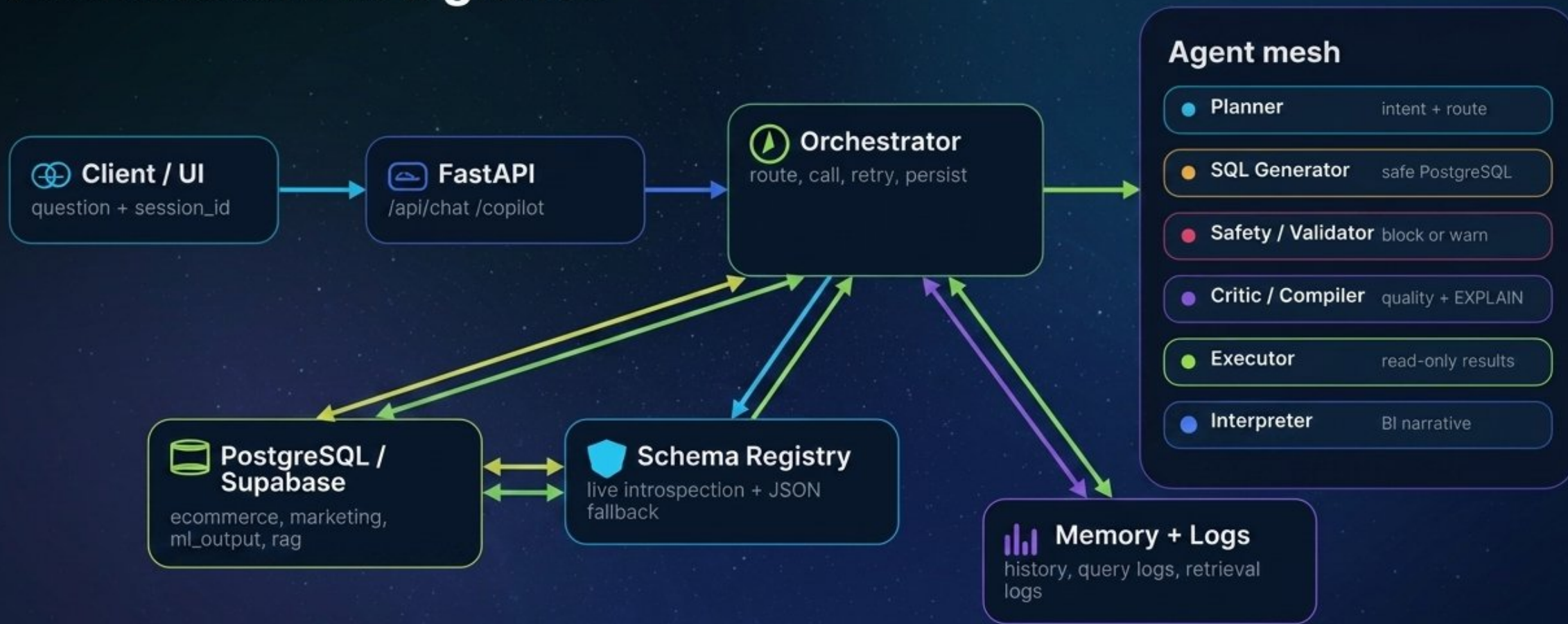
Return answer, SQL audit trail, chart_data and execution steps.



Business outcome: faster leakage triage with safer analytics

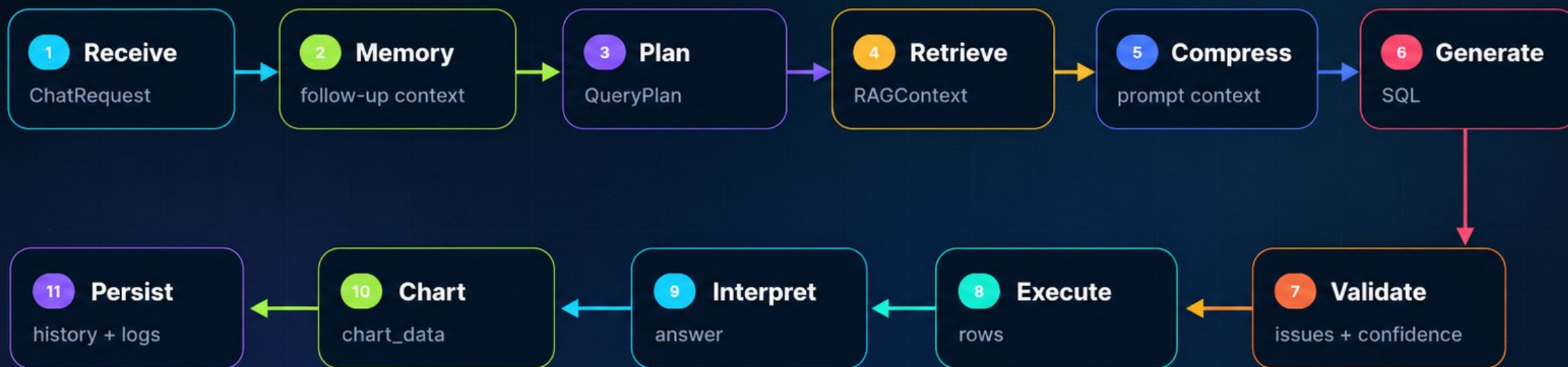
ARCHITECTURE

Architecture at a glance



ARCHITECTURE

How a question moves through the system



The API returns the evidence trail: steps[], sql_used, route, confidence and chart_data

ARCHITECTURE

Why it is hybrid: SQL answers “how much”; RAG explains “why”

SQL lane

Structured facts: revenue, profit, leakage_rate_pct, leakage_revenue, orders, trends and rankings.

materialized views

aggregations

rankings

chart-ready rows

RAG lane

Semantic context: reviews, schema docs, business rules, KPI definitions and SQL examples.

review chunks

business rules

schema docs

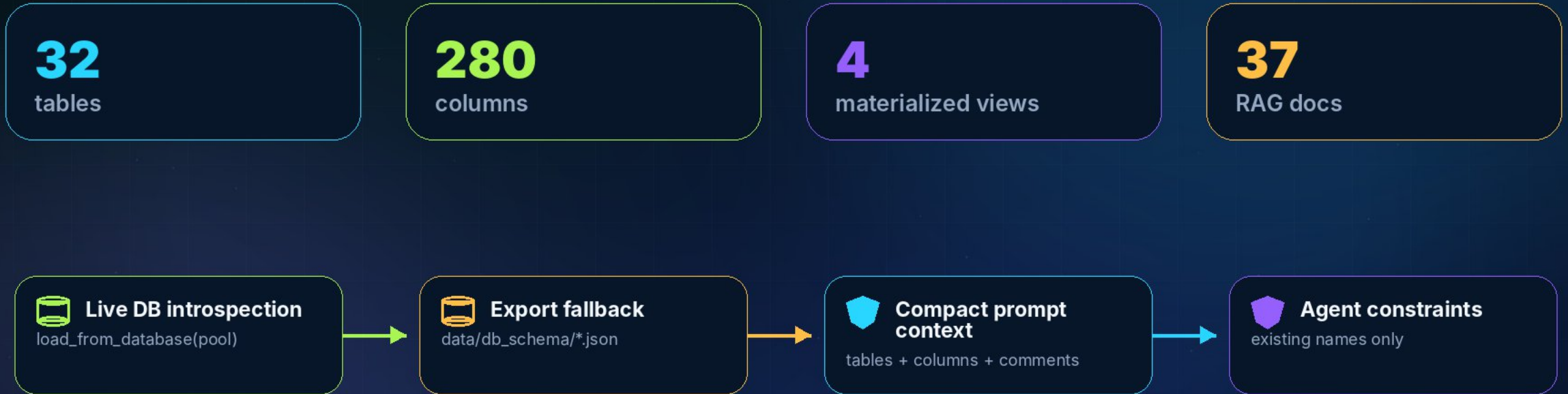
KPI glossary

Grounded answer

facts + evidence + recommendation + chart

ARCHITECTURE

The schema contract prevents hallucination



AGENTS

The agent mesh: small jobs, controlled handoffs

planner_agent.py



Intent, route, tables, metrics

sql_generator.py



Write / repair SELECT SQL

sql_safety.py



Deterministic safety gate

sql_validator.py



Fatal issues + quality warnings

sql_critic.py



SQL/business quality critique

sql_compiler.py



Postgres EXPLAIN validation

query_executor.py



READ ONLY execution + cache

analytics_interpreter.py



Business narrative + cards

Click agent cards in PowerPoint to jump to detailed I/O slides

Story

Business

Architecture

Trust

Demo

AGENTS

Agent I/O matrix

Agent	Input	Job	Output
Planner	message + memory + schema hint	classify + route	QueryPlan
SQL Generator	QueryPlan + SQL context	write / fix SELECT	PostgreSQL SQL
Safety	SQL text	block unsafe shape	SafetyResult
Validator	SQL + plan	fatal vs warning	ValidationResult
Critic	SQL text	business quality checks	warnings + hints
Compiler	SQL + DB connection	EXPLAIN validation	CompileResult
Executor	safe SQL + pool	read-only execution	rows + ms + error
Interpreter	rows + plan + context	BI explanation	answer cards

Story

Business

Architecture

Trust

Demo

AGENTS

Planner Agent: turns language into a QueryPlan



planner_agent.py

Create a structured execution plan

INPUT

- user_message
- session memory context
- compressed schema hints
- intent classifier score

JOB

- Create a structured execution plan

OUTPUT

- intent / route / difficulty
- tables_needed + metrics
- filters / group_by / limit
- chart_policy + language

risk wording → leakage_rate_pct

financial loss → leakage_revenue

case volume → leakage_orders

profit ranking → total_revenue

Story

Business

Architecture

Trust

Demo

AGENTS

SQL Generator: domain semantics become PostgreSQL



sql_generator.py

Generate or repair PostgreSQL SELECT statements

INPUT

- QueryPlan
- compressed SQL context
- schema + SQL examples
- validation / DB error for repair

JOB

- Generate or repair PostgreSQL SELE...

OUTPUT

- single SELECT statement
- schema-qualified tables
- materialized-view preference
- fixed SQL on retry

SQL writer prompt

CRITICAL CONTRACT

ONLY existing tables / columns
 NEVER SELECT *
 NEVER DML / DDL
 ALWAYS LIMIT when useful
 ROUND numeric outputs

Dual ranking -> independent ROW_NUMBER()

AGENTS

Safety + Validator: separate fatal risk from quality warnings



sql_safety.py

Deterministic read-only safety

INPUT

- raw SQL
- sqlglot AST parse
- allowed schemas list

JOB

- Deterministic read-only safety

OUTPUT

- SafetyResult
- blocked DML / DDL
- blocked multi-statement
- outer LIMIT helper



sql_validator.py

Validation result and confidence

INPUT

- SQL
- plan
- deterministic safety result

JOB

- Validation result and confidence

OUTPUT

- ValidationResult
- fatal issues
- warnings
- optimized_sql optional

Design principle: fatal = unsafe; warning = improve but do not break valid analytics

Critic + Compiler: quality checks meet real PostgreSQL



sql_critic.py

Non-fatal business/SQL critique

INPUT

- SQL text
- domain heuristics

JOB

- Non-fatal business/SQL...

OUTPUT

- warnings
- hints
- duplicate join signals
- ranking / payment cautions



sql_compiler.py

Validate with Postgres EXPLAIN

INPUT

- asyncpg connection
- SQL
- timeout_ms

JOB

- Validate with Postgres EXPLAIN

OUTPUT

- CompileResult
- ok / error
- EXPLAIN-based validation
- no data mutation

AGENTS

Query Executor: safe execution and stable response time



query_executor.py

Execute safe SQL and return rows

INPUT

- asyncpg pool
- validated SQL
- use_cache flag
- rag.sql_guard rules

JOB

- Execute safe SQL and return rows

OUTPUT

- rows
- execution_ms
- error message
- was_truncated flag

cache key

sha256(sql)

guard

rag.sql_guard

transaction

READ ONLY

compile

EXPLAIN

cap

outer LIMIT

rollback

safe cleanup

AGENTS

Analytics Interpreter: rows become a business answer



analytics_interpreter.py

Create grounded BI narrative

INPUT

- user message
- SQL rows
- QueryPlan
- compressed analytics context

JOB

- Create grounded BI narrative

OUTPUT

- answer markdown
- normalized section headers
- deterministic trend answers
- Arabic/English handling

Frontend card contract

Key Finding

The main quantified insight.

Business Impact

Why this matters commercially.

Evidence

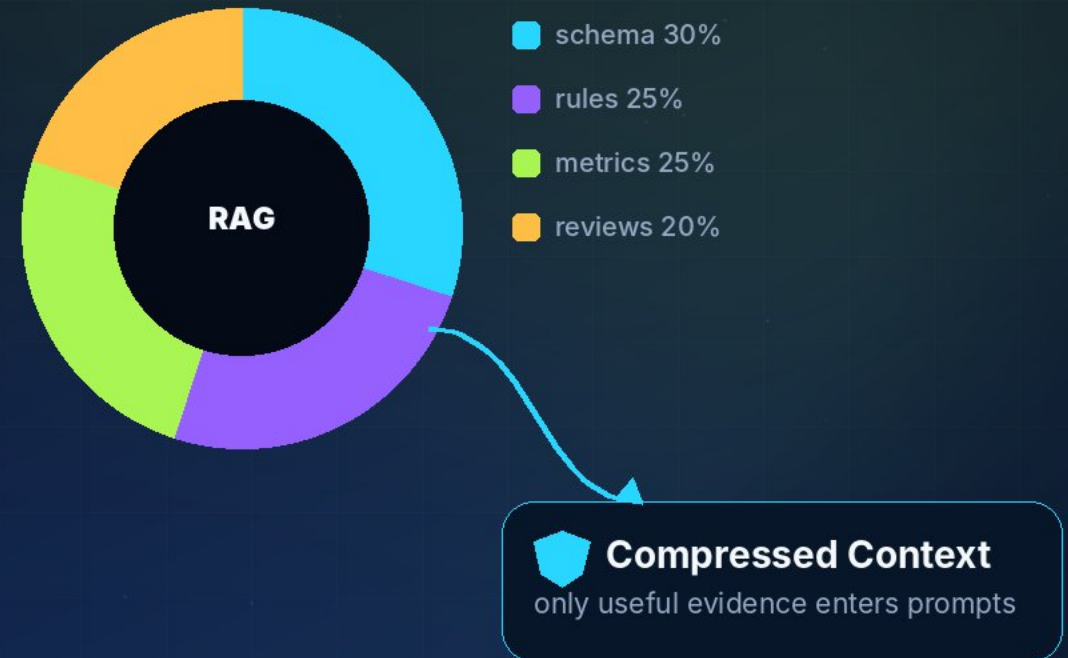
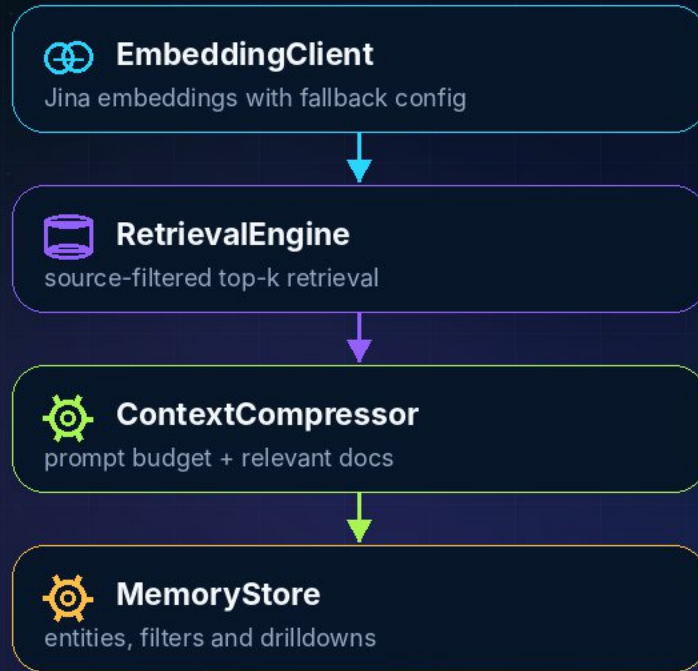
Rows and metrics that support it.

Recommendation

Next action for the business.

ARCHITECTURE

Retrieval and memory keep the answer grounded



ARCHITECTURE

Chart grammar: visuals are selected deterministically

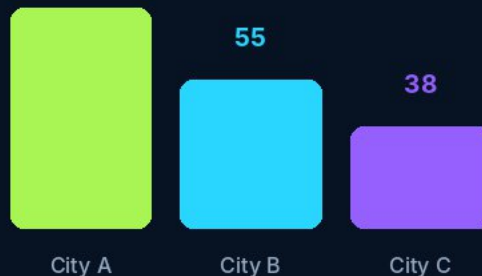
 **Time trend**
line chart

 **Ranking**
bar chart

 **Share / distribution**
doughnut

 **Lookup / KPI**
table or KPI card

Ranking example



Trend example



chart_data contract

```
{
  type: "bar" | "line" | "doughnut" | "table",
  labels: [...],
  datasets: [{ label, data, unit, value_format }]
}
```


TRUST

Trust stack: the answer is defended in layers

Planner constraints

Schema registry

SQL writer contract

Safety AST parse

Validator + critic

Postgres EXPLAIN

READ ONLY execution



Blocked examples

DML / DDL, multiple statements, unknown schemas, unsafe table access



Allowed analytics

CTEs, windows, UNION, rankings, aggregations and complex SELECTs

Story

Business

Architecture

Agents

Demo

ARCHITECTURE

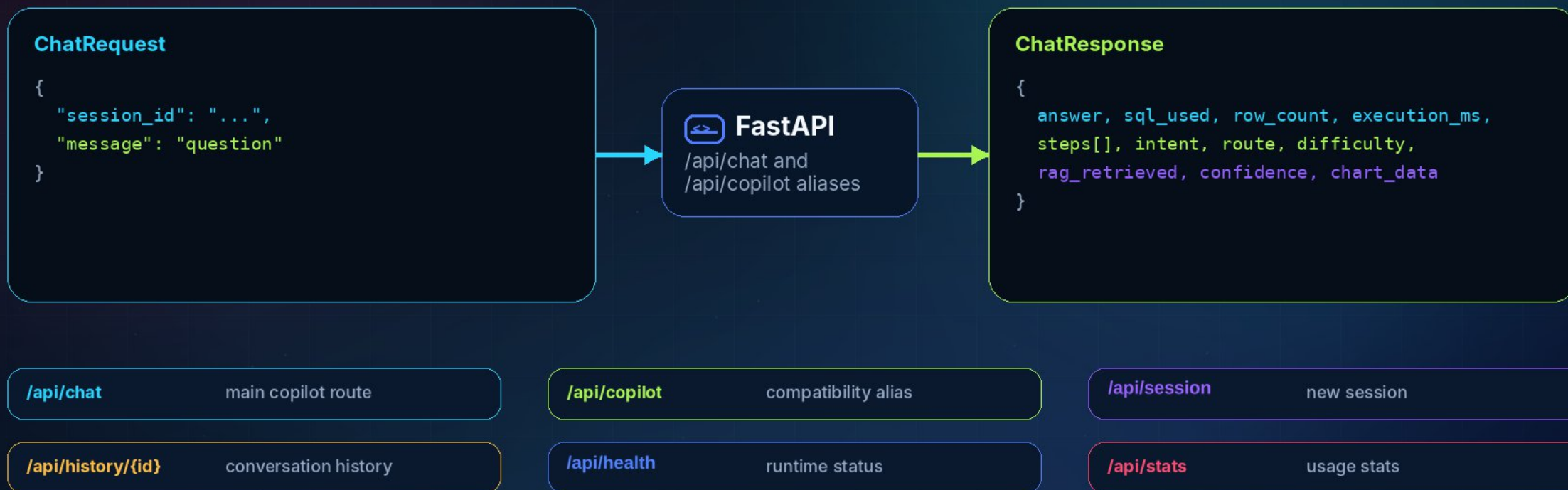
Adaptive fallback: analytics still answers when one source is empty



Cache TTL: source diagnostics are cached for faster repeated questions

ARCHITECTURE

API contract: frontend gets answer + proof + visual payload



DEMO

Example walkthrough: profit + loss ranking

User question

"Most profitable city and city with highest financial loss?"

Planner output

```
route = sql_only
difficulty = complex
metrics = [total_revenue, leakage_revenue]
source = ml_output.mv_leakage_dashboard
rule = independent ROW_NUMBER()
```

Chart result



Plan

controlled handoff

Generate SQL

controlled handoff

Validate

controlled handoff

Execute

controlled handoff

Explain

controlled handoff

Story

Business

Architecture

Agents

Trust

BUSINESS

Business questions the backend can answer

Leakage dashboard



Where is revenue at risk?

Seller risk



Which sellers need action?

Marketing ROI



Which campaigns convert to profit?

Web analytics



How sessions relate to revenue?

Customer reviews



What do negative reviews reveal?

Trend analysis



What changed month over month?

Outputs: answer cards + SQL audit + row counts + chart payload + recommendations

Story

Architecture

Agents

Trust

Demo

DEMO

Interactive navigation map

Story

why it exists

Go to 02

Business

business value + use cases

Go to 03

Architecture

pipeline, RAG, schema, API

Go to 04

Agents

agent I/O and responsibilities

Go to 08

Trust

safety, tests and execution

Go to 18

Demo

example walkthrough

Go to 21

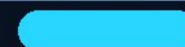
Story

Business

Architecture

Agents

Trust



BUSINESS

The core idea: governed analytics, not generic chat

